

Manejo de memoria en C

Estructuras de Datos

Carlos A Delgado S

Pontificia Universidad Javeriana Cali

Agosto 2026

Objetivos de aprendizaje I

Al finalizar esta sesión, el estudiante podrá

- Explicar cómo organiza la memoria un programa en C: direcciones, regiones y tamaños de los tipos.
- Leer y escribir una variable a través de su dirección con los operadores `&` y `*`.
- Reservar memoria en ejecución con `malloc`, verificarla y devolverla con `free`. ← Java Garbage Collector
- Reconocer una fuga de memoria y un puntero colgante en un programa ajeno.

Objetivos de aprendizaje II

Objetivos de Aprendizaje del curso involucrados

- OA1: apropiar conocimientos de computación mediante el diseño e implementación de soluciones a problemas pequeños.
- OA2: diseñar e implementar soluciones usando las herramientas del lenguaje.

Referencia bibliográfica

R. Thareja, *Data Structures Using C*, Oxford University Press, 2018. Capítulo 6: sección 6.1 (*Understanding the Computer's Memory*), la introducción a punteros y la sección de asignación dinámica de memoria.

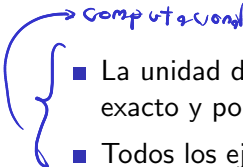
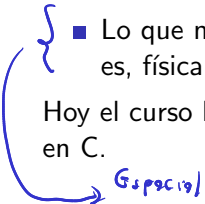
Contenido I

- 1 El problema del servidor cansado
- 2 La memoria por dentro
- 3 Dónde vive cada variable
- 4 La dirección como valor
- 5 Memoria en ejecución
- 6 Lo que sale mal
- 7 Ejercicios
- 8 Ejercicio de cierre
- 9 Resumen y referencias
- 10 Para practicar en casa

Contenido I

- 1 El problema del servidor cansado
- 2 La memoria por dentro
- 3 Dónde vive cada variable
- 4 La dirección como valor
- 5 Memoria en ejecución
- 6 Lo que sale mal
- 7 Ejercicios
- 8 Ejercicio de cierre
- 9 Resumen y referencias
- 10 Para practicar en casa

Lo que dejó la notación O

- 
- Computational
- La unidad de complejidad quedó completa: contar el costo exacto y ponerle techo con la notación O y sus testigos c y k .
 - Todos los ejemplos recorrían arreglos: `datos[i]`, de la posición 0 a la $n - 1$.
- 
- Espacio
- Lo que no hemos preguntado: ¿dónde están esos datos? ¿Qué es, físicamente, `datos[i]`?

Hoy el curso baja a la máquina: cómo usa la memoria un programa en C.

El servidor cansado I

Un servidor de la universidad lleva tres semanas prendido. Cada día responde más lento, y hoy por la mañana dejó de responder. Se reinicia, y vuelve a andar perfecto. Nadie cambió el código.

¿Qué se estaba agotando?

La respuesta está al final de la clase. Para llegar a ella toca entender primero qué le presta la máquina a un programa cuando corre.

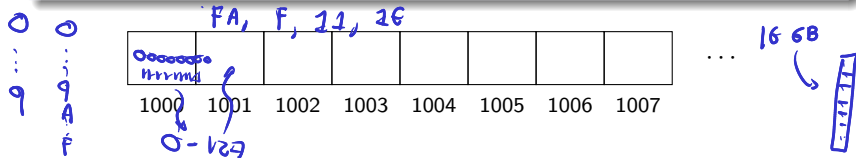
Contenido I

- 1 El problema del servidor cansado
- 2 La memoria por dentro**
- 3 Dónde vive cada variable
- 4 La dirección como valor
- 5 Memoria en ejecución
- 6 Lo que sale mal
- 7 Ejercicios
- 8 Ejercicio de cierre
- 9 Resumen y referencias
- 10 Para practicar en casa

Qué es la memoria I

Definición (Thareja, sección 6.1)

La memoria principal es una secuencia de celdas de un byte. Cada celda tiene un número que la identifica, su *dirección*. Las direcciones son consecutivas: 0, 1, 2, ...



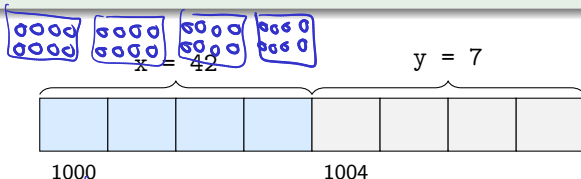
Una ciudad con una sola calle larguísima: cada casa guarda un byte y tiene su número. Para la traza usamos direcciones inventadas y pequeñas (1000, 1004); la máquina real las escribe en hexadecimal.

Una variable ocupa celdas I

int: 32 bits $\rightarrow$ 4 bytes = 8 bits

```
int x = 42; int y = 7;
```

Un int ocupa 4 bytes. Si x empieza en la dirección 1000, ocupa las celdas 1000 a 1003.



- La *dirección* de `x` es la de su primera celda: 1000.
- `y` viene justo después: empieza en 1004.
- El compilador reparte las celdas; el programa las usa.

¿Cuánto ocupa cada tipo? I

El operador sizeof responde en bytes:

```
#include <stdio.h>

int main(void) {
    printf("char:    %zu byte\n", sizeof(char));
    printf("int:     %zu bytes\n", sizeof(int));
    printf("double: %zu bytes\n", sizeof(double));
    printf("int *:    %zu bytes\n", sizeof(int *));
    return 0;
}
```

Salida en la máquina del curso: 0—129

```
char:    1 byte
int:     4 bytes → 0000 --- 0000
double:  8 bytes
int *:   8 bytes ← -Direccion 64 bits
                RAM
```

¿Cuánto ocupa cada tipo? II

La última línea es nueva: hay un tipo cuyo valor es una dirección.
Llegamos a él en unos minutos.

Ahora usted !

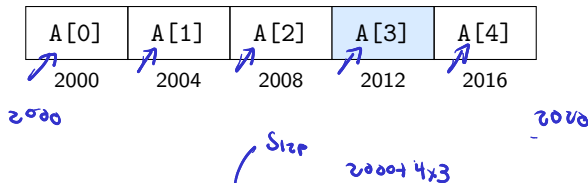
El arreglo `int A[5]` empieza en la dirección 2000.

En parejas: ¿en qué dirección empieza `A[3]`? ¿Y `A[4]`?

Ahora usted: respuesta I

$A[i]$

Cada casilla ocupa 4 bytes y van pegadas:



dirección de $A[i] = 2000 + 4i$ $A[3] \rightarrow 2012$, $A[4] \rightarrow 2016$.

Por eso $A[i]$ cuesta lo mismo con $i = 0$ que con $i = 999$: es una multiplicación y una suma, no un recorrido. La cuenta de la semana pasada tiene aquí su porqué.

$A[i] \quad \Theta(1)$
c o f

La máquina real I

El mismo experimento, preguntándole al programa dónde puso cada cosa (el formato %p imprime direcciones):

```
int x = 42;  
int y = 7;  
int A[3] = {10, 20, 30};  
  
printf("x vale %d y vive en %p\n", x, (void *) &x);
```

16 5B

Una corrida en la máquina del curso:

x vale 42 y vive en 0x7ffed70e6c50

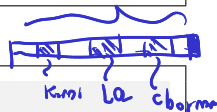
← RAM

y vale 7 y vive en 0x7ffed70e6c54

A[0] vale 10 y vive en 0x7ffed70e6c5c

A[1] vale 20 y vive en 0x7ffed70e6c60

A[2] vale 30 y vive en 0x7ffed70e6c64



5c, 5d, 5e, 5f, 60

60, 61, 62, 63

64, 65, 66, 67

La máquina real II

- De x a y hay 4 bytes; entre casillas de A, también (en hexadecimal: c5c, c60, c64).
- Otra corrida puede dar otras direcciones: el sistema ubica al programa donde quiera. Las distancias se conservan.

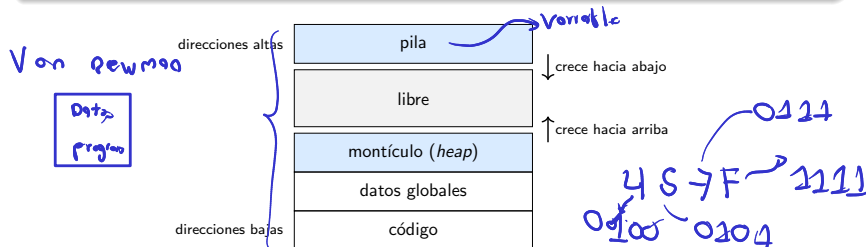
Contenido I

- 1 El problema del servidor cansado
- 2 La memoria por dentro
- 3 Dónde vive cada variable**
- 4 La dirección como valor
- 5 Memoria en ejecución
- 6 Lo que sale mal
- 7 Ejercicios
- 8 Ejercicio de cierre
- 9 Resumen y referencias
- 10 Para practicar en casa

Las regiones de un programa I

El mapa (Thareja, cap. 6, *Memory Allocation in C Programs*)

La memoria de un programa en ejecución se reparte en regiones: el *código*, los *datos globales*, la *pila* (locales de cada función) y el *montículo* (memoria pedida en ejecución).



Hasta hoy solo hemos usado dos: el código y la pila.

Una variable de cada región I

```
#include <stdio.h>
#include <stdlib.h>

int global = 5;

int main(void) {
    int local = 10;
    int *dinamico;
    int estado = 0;

    dinamico = malloc(sizeof(int));
    if (dinamico == NULL) {
        printf("No hay memoria disponible\n");
        estado = 1;
    } else {
        *dinamico = 15;
        printf("global:  %d en %p\n", global, (void *)
            ↪ &global);
    }
}
```

✓ Reservar 4 bytes
32 bits

Una variable de cada región II

```
    printf("local:    %d en %p\n", local, (void *)  
        ↪ &local);  
    printf("dinamico: %d en %p\n", *dinamico, (void *)  
        ↪ dinamico);  
    free(dinamico);  
    dinamico = NULL;  
}  
return estado;  
}
```

El programa usa malloc y *, que se definen en las dos secciones que siguen; por ahora, interesa la salida.

Una variable de cada región: la salida l

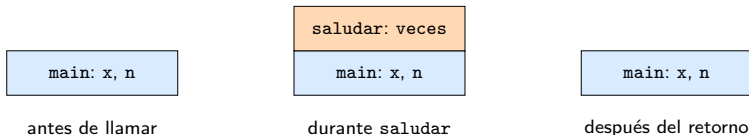
```
global: 5 en 0x564e1c944038
local: 10 en 0x7ffed4709dc8
dinamico: 15 en 0x564e58e36010
```

Variable	Región	Dirección
global	datos globales	0x564e1c94...
local	pila	0x7ffed470...
el int nuevo	montículo	0x564e58e3...

Tres direcciones en barrios distintos de la ciudad. La pila queda lejísimos de las otras dos: arriba del todo, como en el mapa.

La pila respira I

Cada llamada a una función apila un *marco* con sus variables locales; el retorno lo desapila. Tres momentos de un programa donde `main` llama a `saludar`:



- `veces` nace al entrar a `saludar` y muere al retornar: su celda se reutiliza en la siguiente llamada.
- Por eso una función no puede “dejarle” a otra un dato en una variable local. Guardar algo que sobreviva a la función pide otra región: el montículo.

Contenido I

- 1 El problema del servidor cansado
- 2 La memoria por dentro
- 3 Dónde vive cada variable
- 4 La dirección como valor**
- 5 Memoria en ejecución
- 6 Lo que sale mal
- 7 Ejercicios
- 8 Ejercicio de cierre
- 9 Resumen y referencias
- 10 Para practicar en casa

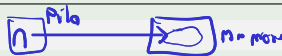
El operador & |

Definición (Thareja, cap. 6)

Para una variable v , la expresión $\&v$ es la dirección donde v vive. Se lee “la dirección de v ”.

Ya lo veníamos usando

```
scanf("%d", &n);
```



`scanf` no recibe a n, recibe su dirección para poder escribir el valor leído en las celdas de n . El `&` de las primeras clases era esto.

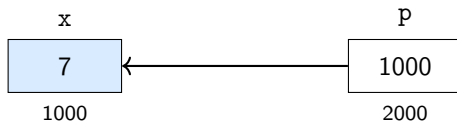
Si una dirección es un valor, se puede guardar en una variable. ¿De qué tipo?

El puntero I

Definición (Thareja, cap. 6)

Un *puntero* es una variable cuyo valor es la dirección de otra variable. `int *p` declara un puntero a `int`: `p` guarda la dirección de un entero.

```
int x = 7;  
int *p = &x;    /* p guarda la direccion de x */
```



- `p` es una variable como cualquiera: vive en 2000 y su valor es 1000. `*p` se lee “lo que hay donde apunta `p`”: leer `*p` da 7; asignar `*p = 9` cambia a `x`.

¿Qué imprime? I

En parejas: siga el programa línea a línea y anticipe la salida.

```
#include <stdio.h>

int main(void) {
    int x = 3;
    int y = 8;
    int *p;

    p = &x;           /* p guarda la direccion de x */
    *p = 7;           /* escribe a traves de p */
    p = &y;           /* p ahora apunta a y */
    *p = *p + 1;

    printf("x = %d, y = %d\n", x, y);
    return 0;
}
```

¿Qué imprime?: la traza I

Con x en 1000 y y en 1004:

Línea	x	y	p
declaraciones	3	8	?
p = &x;	3	8	1000
*p = 7;	7	8	1000
p = &y;	7	8	1004
*p = *p + 1;	7	9	1004

x = 7, y = 9

- *p = 7 cambió a x sin nombrarla: el puntero es un segundo nombre para la misma celda.
- Tras p = &y, el mismo *p habla de otra variable. Qué toca *p depende del valor de p en ese momento.

¿Y si no se inicializa? I

En parejas: ¿qué hace este programa?

```
int main(void) {  
    int *p;  
    *p = 5;  
    return 0;  
}
```

¿Y si no se inicializa?: respuesta I

El compilador lo ve venir:

```
sin_inicializar.c: In function 'main':  
warning: 'p' is used uninitialized [-Wuninitialized]  
    3 |     *p = 5;  
      |     ~~~~~
```

p contiene basura: la escritura cae en una dirección cualquiera. A veces el programa revienta de inmediato; a veces corrompe un dato y falla mucho después, lejos del error.

Regla de la casa

Todo puntero nace apuntando a algo concreto o a NULL, la dirección “a ninguna parte”. Un puntero declarado y sin asignar no se usa. Por eso compilamos siempre con `-Wall -Wextra`: ese aviso es el error anunciado.

Contenido I

- 1 El problema del servidor cansado
- 2 La memoria por dentro
- 3 Dónde vive cada variable
- 4 La dirección como valor
- 5 Memoria en ejecución**
- 6 Lo que sale mal
- 7 Ejercicios
- 8 Ejercicio de cierre
- 9 Resumen y referencias
- 10 Para practicar en casa

El tamaño que no se conoce I

Un programa va a leer las temperaturas de la semana... ¿de cuántos días? Eso lo decide el usuario al ejecutar.

```
int datos[1000];    /* y si son 20? y si son 5000? */
```

- Con 20 valores sobran 980 casillas reservadas en vano.
- Con 5000, el programa no tiene dónde ponerlos.
- El arreglo de la pila exige decidir el tamaño al escribir el programa; el problema lo decide al correr.

Se necesita pedir memoria *durante* la ejecución, del tamaño exacto. Para eso existe el montículo.

malloc y free I

Definición (Thareja, cap. 6, *Dynamic Memory Allocation*)

`malloc(k)` reserva un bloque de `k` bytes en el montículo y devuelve su dirección; si no hay memoria, devuelve `NULL`. `free(p)` devuelve el bloque que empieza en `p`. El bloque vive hasta su `free`, sin importar en qué función se pidió.

```
datos = malloc(n * sizeof(int));    /* n enteros */
```

- El tamaño se escribe con `sizeof`: n enteros son `n * sizeof(int)` bytes.
- La primera pregunta después de todo `malloc`: ¿devolvió `NULL`?
- `calloc(n, sizeof(int))` hace lo mismo y además deja el bloque en ceros.

El arreglo del tamaño justo I

Primera mitad: leer la cantidad y reservar.

```
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int n = 0;
    int *datos;
    int i;
    int estado = 0;

    printf("Cuantos valores? ");
    if (scanf("%d", &n) != 1 || n <= 0) {
        printf("Cantidad invalida\n");
        estado = 1;
    } else {
        datos = malloc(n * sizeof(int));
        if (datos == NULL) {
            printf("No hay memoria disponible\n");
        }
    }
}
```



El arreglo del tamaño justo II

```
        estado = 1;  
    } else {
```

Dos verificaciones antes de tocar el bloque: la cantidad es válida y la reserva llegó.

El arreglo del tamaño justo (continuación) I

Segunda mitad: usar el bloque como un arreglo cualquiera y devolverlo.

```
i = 0;
while (i < n) {
    datos[i] = i * i;
    i = i + 1;
}
i = 0;
while (i < n) {
    printf("datos[%d] = %d\n", i, datos[i]);
    i = i + 1;
}
free(datos);
datos = NULL;
}
return estado;
```

Handwritten notes:

- sizeof(int) (with an arrow pointing to the `datos` array)
- $8 \text{ datos} + 4 \times i$ (next to the assignment `datos[i] = i * i;`)
- $\left. \begin{array}{l} \leftarrow \text{libera memoria} \\ \leftarrow \text{libera el puntero} \end{array} \right\} \text{Garbage Collector}$ (next to `free(datos);` and `datos = NULL;`)
- $\leftarrow \text{delete datos; } \leftarrow \text{C++}$ (next to `datos = NULL;`)

El arreglo del tamaño justo (continuación) II

```
}
```

Una corrida con $n = 4$:

```
Cuantos valores? 4
```

```
datos[0] = 0
```

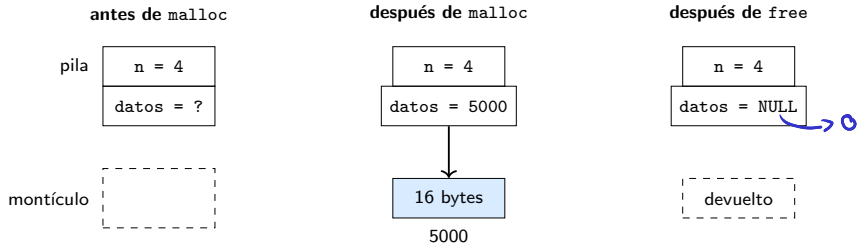
```
datos[1] = 1
```

```
datos[2] = 4
```

```
datos[3] = 9
```

La traza en el mapa l

Tres momentos del programa, con $n = 4$:



- `datos` vive en la pila; el bloque de 16 bytes, en el montículo. El puntero es el hilo que los une.
- `free` devuelve el bloque, no borra a `datos`: por eso lo ponemos en `NULL` a mano, para que no quede apuntando a memoria ajena.

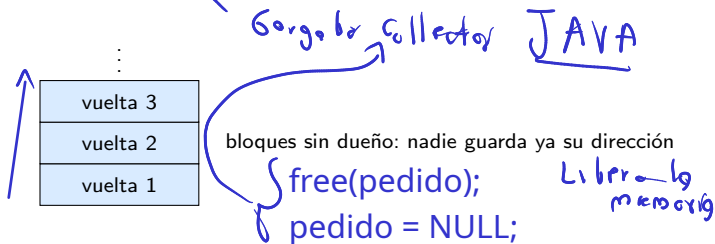
Contenido I

- 1 El problema del servidor cansado
- 2 La memoria por dentro
- 3 Dónde vive cada variable
- 4 La dirección como valor
- 5 Memoria en ejecución
- 6 Lo que sale mal**
- 7 Ejercicios
- 8 Ejercicio de cierre
- 9 Resumen y referencias
- 10 Para practicar en casa

El servidor, resuelto I

El código del servidor hacía, en esencia, esto por cada solicitud:

```
while (i < 1000) {  
    int *pedido = malloc(250 * sizeof(int));  
    if (pedido != NULL) {  
        pedido[0] = i;  
        atendidas = atendidas + 1;  
    }  
    i = i + 1; /* el bloque queda reservado para siempre */  
}
```



El servidor, resuelto II

Fuga de memoria

En cada vuelta, pedido se sobrescribe y la dirección del bloque anterior se pierde: ese bloque no se puede liberar nunca. El montículo se llena poco a poco, el programa se arrastra y al final malloc empieza a devolver NULL. Eso se estaba agotando en el servidor: reiniciar lo “cura” porque el sistema recupera toda la memoria del proceso al morir.

El puntero colgante I

El error simétrico: devolver el bloque y seguirlo usando.

```
p = malloc(sizeof(int));  
if (p == NULL) {  
    estado = 1;  
} else {  
    *p = 42;  
    free(p);  
    printf("%d\n", *p);    /* la memoria ya no es nuestra */  
}
```

p = NULL;

El compilador del curso también lo anuncia:

```
warning: pointer 'p' used after 'free' [-Wuse-after-free]
```

El puntero colgante II

Puntero colgante

Después de `free(p)`, la dirección guardada en `p` apunta a memoria que ya no es del programa: puede haberse entregado a otro `malloc`. Leerla o escribirla es comportamiento indefinido, el mismo veneno del puntero sin inicializar.

Las tres reglas I

El contrato con el montículo

- 1 A todo `malloc` se le pregunta si devolvió `NULL` antes de usar el bloque.
- 2 Todo `malloc` tiene su `free`: quien pide, devuelve.
- 3 Después de `free(p)`, se asigna `p = NULL`: un puntero a ninguna parte es mejor que un puntero a memoria ajena.

Las tres reglas aparecen juntas en cada ejemplo del curso desde hoy. Cuando implementemos listas enlazadas, serán el esqueleto de crear y destruir.

Contenido I

- 1 El problema del servidor cansado
- 2 La memoria por dentro
- 3 Dónde vive cada variable
- 4 La dirección como valor
- 5 Memoria en ejecución
- 6 Lo que sale mal
- 7 Ejercicios**
- 8 Ejercicio de cierre
- 9 Resumen y referencias
- 10 Para practicar en casa

Ejercicio 1 I

En parejas, sin correrlo: ¿qué imprime? Siga la tabla de la clase, una fila por línea, con direcciones inventadas.

```
int a = 10;  
int b = 20;  
int *p = &a;  
int *q = &b;  
  
*p = *q;  
q = p;  
*q = 30;  
  
printf("a = %d, b = %d\n", a, b);
```

Ejercicio 1: respuesta I

Con a en 1000 y b en 1004:

Línea	a	b	p	q
declaraciones	10	20	1000	1004
*p = *q;	20	20	1000	1004
q = p;	20	20	1000	1000
*q = 30;	30	20	1000	1000

a = 30, b = 20

- *p = *q copia el *valor*: 20 viaja a a, y los punteros no se mueven.
- q = p copia la *dirección*: desde ahí los dos hablan de a, y b no vuelve a cambiar.

Ejercicio 2 I

Este fragmento compila sin avisos. En parejas: tiene dos errores de memoria, uno de cada sección de hoy. Encuéntrelos.

```
int *a = malloc(n * sizeof(int));  
int *b = malloc(n * sizeof(int));  
  
a[0] = 1;  
b[0] = 2;  
b = malloc(2 * n * sizeof(int));  
b[0] = 3;  
  
free(a);  
free(b);
```

$\text{int} = 32 \text{ bits} = 4 \text{ bytes}$

$\text{long} = 64 \text{ bits} = 8 \text{ bytes}$

$\text{double} = 64 \text{ bits} = 8 \text{ bytes}$

$\text{long long} = 128 \text{ bits} = 16 \text{ bytes}$

Ejercicio 2: respuesta I

- Ninguno de los tres `malloc` se pregunta por `NULL`: la escritura `a[0] = 1` puede caer en ninguna parte. Se rompe la regla 1.
- El segundo `malloc` sobre `b` pisa la única dirección del bloque anterior: ese bloque queda sin dueño y su `free` ya no existe. La fuga del servidor, en miniatura. Se rompe la regla 2.

El arreglo: verificar cada reserva y liberar el bloque viejo de `b` (o no pedir el segundo) antes de reasignar el puntero. Los `free` del final, con `b` ya reasignado, solo alcanzan dos de los tres bloques pedidos.

Contenido I

- 1 El problema del servidor cansado
- 2 La memoria por dentro
- 3 Dónde vive cada variable
- 4 La dirección como valor
- 5 Memoria en ejecución
- 6 Lo que sale mal
- 7 Ejercicios
- 8 Ejercicio de cierre**
- 9 Resumen y referencias
- 10 Para practicar en casa

Ejercicio de cierre I

Escriba un programa completo que:

- 1 Lea una cantidad n y reserve espacio para n temperaturas (double).
- 2 Lea las n temperaturas.
- 3 Calcule el promedio y cuente cuántas lo superan.
- 4 Imprima ambos resultados y devuelva la memoria.

Con las verificaciones y las tres reglas de la sesión. ¿Por qué este problema no se puede resolver leyendo los valores de a uno, sin guardarlos?

Contenido I

- 1 El problema del servidor cansado
- 2 La memoria por dentro
- 3 Dónde vive cada variable
- 4 La dirección como valor
- 5 Memoria en ejecución
- 6 Lo que sale mal
- 7 Ejercicios
- 8 Ejercicio de cierre
- 9 Resumen y referencias**
- 10 Para practicar en casa

Lo que queda de la sesión

- La memoria es una secuencia de celdas numeradas; cada variable ocupa las suyas y su dirección es la primera.
- El programa se reparte en regiones: código, datos globales, pila (locales, muere con la función) y montículo (vive hasta su `free`).
- `&v` es la dirección de `v`; un puntero la guarda y `*p` usa la celda apuntada.
- `malloc` reserva en ejecución el tamaño justo; `free` lo devuelve.
- Los dos errores con nombre propio: la fuga (reservar y no devolver) y el puntero colgante (devolver y seguir usando).

Resumen II

Lo que sigue: punteros y arreglos a fondo, y arreglos bidimensionales.

Referencias I



R. Thareja. *Data Structures Using C*. Oxford University Press, 2018. Capítulo 6: sección 6.1, introducción a punteros y asignación dinámica de memoria.



N. Kalicharan. *Data Structures in C*. 2008. Uso de `malloc`, `sizeof` y `free`.

Contenido I

- 1 El problema del servidor cansado
- 2 La memoria por dentro
- 3 Dónde vive cada variable
- 4 La dirección como valor
- 5 Memoria en ejecución
- 6 Lo que sale mal
- 7 Ejercicios
- 8 Ejercicio de cierre
- 9 Resumen y referencias
- 10 Para practicar en casa

Propuesto 1 I

Escriba un programa que lea n , reserve un arreglo de n enteros, lo llene con los primeros n impares (1, 3, 5, ...) y lo imprima de la última posición a la primera. Con verificaciones y `free`.

Propuesto 2 I

Este fragmento compila sin avisos y aun así tiene un error de memoria. ¿Cuál es y en qué línea nace?

```
int *mayores = malloc(n * sizeof(int));  
int *todos = malloc(2 * n * sizeof(int));  
if (mayores == NULL || todos == NULL) {  
    estado = 1;  
} else {  
    /* ... se usan ambos arreglos ... */  
    mayores = todos;  
    free(mayores);  
    free(todos);  
}
```

0 int

Pistas de los propuestos I

- **Propuesto 1:** el impar i -ésimo es $2i + 1$. El esqueleto es el del arreglo del tamaño justo; solo cambian el llenado y el sentido del recorrido de impresión.
- **Propuesto 2:** cuente los malloc y los free de cada bloque. La asignación mayores = todos pierde la única dirección del primer bloque (fuga), y los dos free llegan al mismo bloque: liberar dos veces es tan grave como usar después de liberar.